

Multi-Agent Architectures That Ship: Five Patterns, Three Roles & Validation Contracts

Assemble agent teams that run for days — by separating who builds from who verifies, defining 'done' before any code exists, and putting the right model in each seat.

For engineers who have built single-agent workflows and now want agent teams that stay coherent over hours or days · 30 July 2026

The one- or two-page version: what matters, the topics it covers, and every term defined. Read the full lesson for the explanations and worked examples.

[Watch the source video](#)[Read the full lesson](#)[Read the condensed version](#)

What matters

- ✓ Five patterns compose every multi-agent system: delegation, creator-verifier, direct communication, negotiation, and broadcast.
- ✓ Direct communication (agents DMin each other with no coordinator) is the one to avoid — state fragments and there is no single source of truth.
- ✓ Separate the agent that builds from the agent that verifies: the author has a cost bias toward its own code working, exactly why humans do code review.
- ✓ A validation contract, written during planning before any code, defines correctness independently of implementation — often hundreds of assertions, each feature owning one or more.
- ✓ Tests written after implementation are shaped by the code rather than the intent, so they confirm decisions instead of catching bugs.
- ✓ Validate behaviour, not just code: a QA-style validator drives the running application through computer use, and that real-world execution — not token generation — consumes most of a mission's wall-clock time.
- ✓ Structured handoffs (what was done, what was left, commands run with exit codes, issues found) are what let the system self-heal at milestone boundaries.
- ✓ Serial execution with targeted read-only parallelism beat parallel workers: coordination overhead ate the speed gains while the error rate rose.
- ✓ Planning, implementation, and validation reward different model strengths, and no single provider is best at all three — being model-agnostic means you are not capped by one family's weakest capability.
- ✓ Keep orchestration in prompts and skills rather than a hard-coded state machine, so every model improvement makes the system better instead of obsolete.

What it covers

01 The bottleneck is attention, not intelligence 0:47

Models can already handle the whole backlog; what runs out is the human bandwidth to supervise them.

02 The five multi-agent communication patterns 1:28

Delegation, creator-verifier, direct communication, negotiation, and broadcast — the primitives every framework is built from.

03 Missions: four patterns in one workflow, three roles 4:12

Orchestrator plans, workers implement, validators verify — an ecosystem communicating through structured handoffs and shared state, not one long agent session.

04 **Validation contracts: define 'done' before writing code** 6:15

Tests written after implementation are shaped by the code rather than the intent, so they confirm decisions instead of catching bugs.

05 **Two validators, adversarial by design** 6:57

A scrutiny validator runs the suite and spawns per-feature code review; a user-testing validator drives the live application like a QA engineer.

06 **Structured handoffs and self-healing** 8:19

A worker does not just report 'done' — it writes down what happened, and the system corrects course at milestone boundaries.

07 **Serial execution beats parallelism** 9:00

Ten agents did not give ten times the throughput — they conflicted, duplicated work, and made inconsistent decisions until coordination overhead ate the gains.

08 **The right model in each seat** 11:03

Planning wants slow careful reasoning, implementation wants fast code fluency, validation wants precise instruction following — and no one provider is best at all three.

09 **Beating the bitter lesson: prompts, not state machines** 14:27

Keep orchestration logic in text so the next model release improves the system instead of obsoleting it.

10 **Production reality, and what it unlocks** 13:06

A Slack clone built end to end: 60% of time and tokens on implementation, validation never passing first time, and half the final codebase being tests.

Glossary

Mission — A long-running multi-agent workflow: you approve a plan, then an ecosystem of agents executes it for hours or days via structured handoffs and shared state.

Creator-verifier — Separating the agent that builds something from the agent that checks it, because the author is biased toward its own work succeeding.

Negotiation — Agents coordinating over a shared resource such as an API or a region of the codebase, ideally finding a positive-sum outcome.

Orchestrator — The planning role: interrogates requirements and produces features, milestones, and the validation contract.

Validator — The verification role: an agent that has never seen the code, checking it against the contract — adversarial by design.

Scrutiny validator — The static validator: test suite, type checking, lints, plus a dedicated code review agent per completed feature.

Delegation — One agent spawning another to do a scoped piece of work and return a result — the simplest multi-agent pattern.

Direct communication — Agents messaging each other with no central coordinator. Avoided here because state fragments and no single source of truth exists.

Broadcast — One agent distributing information to many — status, new context, shared constraints — to keep a long run coherent.

Worker — The implementation role: takes one feature with clean context, implements it, and commits so the next worker inherits a working codebase.

Validation contract — A set of assertions defining correctness, written during planning before any code, with each feature assigned the assertions it must satisfy.

User testing validator — The behavioural validator: spawns the running application and drives it via computer use, checking whole functional flows.

Structured handoff — A worker's written record of what it completed, what it left undone, the commands and exit codes, issues found, and whether it followed procedure.

Droid whispering — The skill of modelling how different models behave and fail, and deliberately assigning each to the role it suits.

Milestone boundary — The checkpoint after a group of features where validators run, handoffs are reviewed, and corrective work is scoped.

Bitter lesson — The observation that general methods leveraging more compute tend to beat hand-engineered structure — hence keeping orchestration in prompts rather than code.

Explanations are AI-generated. Check anything you intend to rely on.